Dimension scaled priors and dimension reduction for Bayesian optimzation

Kasper Notebomer (s3914038)
Nael Belhaj Haddou (s4431278)

Abstract

Bayesian optimization is a widely used black-box optimization technique but it is ill-adapted to high dimension search spaces. Multiple ways to tackle high dimension Bayesian optimization have been published, including dimensionality reduction techniques, variable selection techniques and dimensionality scaled prior. Combining these methods may combine their advantages, allowing for fast convergence and good scaling with dimensionality increases. To test this idea, different algorithms reliant on dimension reduction or variable selection are combined with d-scaled priors and are tested on popular black box optimization benchmarking functions. For most algorithms tested, adding d-scaled priors does not seem to significantly improve performances and simple d-scaled priors without any additions consistently outperforms them.

1 Introduction

Bayesian optimization is an optimization technique that aims to maximize or minimize a black box function efficiently. It achieves this by using a surrogate model to generate a model of the underlying function, and then uses this model to efficiently suggest a new point to evaluate. The most used type of surrogate model is known as a Gaussian Process (GP). The benefit of this model is that, once conditioned on a set of observation, it can predict both the expected mean and the uncertainty in the performance of a given point. To quantify the likelihood of a new configuration improving over the current best known and acquisition function is used on the prediction of the surrogate. One of the most common acquisition function families is the expected improvement function. This function quantifies the improvement in terms of likelihood of improvement over the incumbent and the magnitude of the improvement. Maximizing the acquisition function will generate a new point that, under the assumptions of the acquisition function, is most likely to improve over the best known observation.

One of the largest issues with BO is the curse of dimensionality. As the dimensionality of the search space increases, it becomes exponentially harder to find the optimum in the search space. Different methods have been suggested to deal with this phenomenon, all focusing on different techniques. Two of these families are linear/non-linear embedding and variable selection. With embedding, the aim to embed the higher dimensional space into a lower dimensional subspace and optimize over this subspace, thereby decreasing dimensionality and complexity. Variable selection on the other hand tries to quantify active variables and direct search toward these dimension, while "deactivating" non-active variables, thereby reducing the complexity of the search.

A recent paper proposes the idea that the main issue with high dimensional Bayesian optimization is not necessarily the dimensionality of the problem, but the assumptions made by the hyperparameter

choices placed on the GP. The main idea is that the average distance in a unit cube scales in the scales in $\sqrt{d}$ as the dimensionality (d) increases. If this is not accounted for in the lengthscale of the GP, high dimensionality will cause the fitted model to have many regions where the GP reverts to the prior. This behavior is unwanted, as it will lead the model to either over explore as there are an almost infinite number of points where the variance is maximum (prior), or it will over explore around the DoE points, as these are the only areas where the model actually knows anything. In high dimensions the problem with low lengthscales is mainly the fact that the GP can be conditioned on many points, without any of them actually interacting with each other. Seeing as this is the case, the GP should also be trained with hyperparameters that compensate for this behavior. The method proposed by Hvarfner et al. [7] places a hyperprior on the lenghtscale of the kernel and use automatic relevance detection (ARD). ARD means that every dimension has a separate lengthscale parameter. The lengthscales are then estimated using maximum a posteriori (MAP) estimation. To combat the growth of the distances as the dimensionality, the proposed method scales the mean of prior on the lengthscales by the square root of the dimensionality, resulting in $\ell \propto \sqrt{d}$. The idea being that this increase in the lengthscale should offset the growth in dimensionality. It was found that this relatively simple idea performs quite well. It was however noted that, when looking at the lengthscales, the technique did not seem to be able to accurately identify known active variables for certain problems.

All of this leads to two main questions. The first is if this enhancement is something that can be used to enhance the performance of other methods that focus on dimension reduction and variable selection. The second one is if dimension reduction and variable selection techniques could help improve the performance by guiding search towards active variables, either by variable selection or by optimizing in an active subspace.

2 Methods

2.1 Theory

We will refer to the base method proposed by [7] as DSP (dimensionality scaled priors). The specific implementation was made as close to the original method proposed by the authors. This meant the inputs to the GP were normalized to a range of $[0, 1]$, and the outputs where z-score standardized to have a mean of 0 and a standard deviation of 1. The radial basis function (RBF) was used as the kernel function, and qLogNoisyExpectedImprovement was used for the acquisition function. The lengthscale prior that was used was a lognormal distribution with $\mu = \sqrt{2} + \frac{\log\sqrt{d}}{2}$ and $\sigma = \sqrt{3}$ resulting in 1.

$$\ell_i = \mathcal{LN}(\sqrt{2} + \frac{\log\sqrt{d}}{2}, \sqrt{3}) \tag{1}$$

The main algorithms for dimension reduction/embedding that were tested were REMBO, HeSBO, PCA-BO and GTBO.

Random Embeddings Bayesian Optimization (REmBO) [11] is an algorithm that attempts to tackle the curse of dimensionality using linear embeddings. For a problem of dimensionality D it creates a matrix $A \in \mathbb{R}^{D\times d}$ to map a lower dimension embedding space to problem's dimensionality. The matrix is built such that $A_{i,j} \sim N(0, 1)$. After building the embedding the matrix the authors bind the search space to the range $[-\sqrt{d}, \sqrt{d}]$. After this, regular Bayesian optimization is performed on the embedded

space. When choosing a point x to evaluate, $f(xA)$ is evaluated instead. Then $(x, f(xA)$ is added to the kernel's data. In principle this embedded search space will represent the function's landscape. However, the biggest drawback is the lack of a way to efficiently identify d and if d is too large, leading back to the same problem of high dimensional Bayesian optimization.

Hashing-enhanced subspace Bayesian Optimization (HeSBO) [8] is a linear embedding technique that relies on the idea that for a high dimensional space D, there exists a mapping to a lower dimensional subspace of size d. This lower dimensional subspace is referred to as the active subspace, and the idea is that points outside this subspace won't cause changes to the objective function value. HeSBO constructs this subspace using the principles of count-sketch [3] and by making use of two hashing functions, $h : [D] \to [d]$ and $\sigma : [D] \to \{-1, 1\}$. These two function represent an embedding matrix S' that maps a vector Y from the lower dimensional space to the higher dimensional space X. h picks a singular non-zero entry for each vector/dimension (D) in S', while σ determines the sign on this entry. These hashing functions are determined at random. In simple terms, using this algorithm, every entry in the lower dimensional vector represents a linear combination of variables from the higher dimensional space.

Another dimension reduction technique that was implemented was PCA-BO [9]. This method again relies on the idea of the existence of a lower dimensional subspace to make optimization more efficient. PCA-BO, as the name suggest, makes use of PCA to build the embedded space. In theory, the principal component found by PCA should align with directions where the function value shows high variance. To this end, a weighing scheme is proposed by the authors, where points with more optimal function values are given larger weights/importance. The specific scheme is a rank-based weighting scheme. Here, the weight of an observed point is decided by its rank, which is calculated using equation 2. Here $\{r\}_{i=1}^{n}$ denotes the ranking of the observed point based on their function value. $\widetilde{w}_i$ is the unnormalized weight, which is normalized to $w_i = \frac{\widetilde{w}_i}{\Sigma \widetilde{w}_i}$.

$$\widetilde{w}_i = ln(n) - ln(r_i) \tag{2}$$

After fitting PCA, the dimensionality can be reduced by only keeping a subset of the principal components. This is done by ordering the principal components by the amount of variance explained by them, and only keeping the r components that combined explain α percent of the total variance. For our experiments, we set α to 95%, as also used by the authors in their respective experiments. The GP can now be fitted on the lower dimensional subspace. The acquisition function used for this model was penalized expected improvement (PEI). This had to be used over EI due to the way in which PCA transforms the space, meaning that the original box constraints are not equal to the ones in the reduced space. Therefore, a new search space constraint has to be set for the lower space, but in such a way that it is still likely to include all points on the original domain. This means the search domain in the reduced space will likely also contain point which fall outside the domain of the original function. PEI gives a penalty to points in the reduced search space that would fall outside the original functions' domain, thereby effectively discarding them during optimization of the acquisition function. For the exact specifications and calculations for the constraint in the reduced space we refer to the original paper by Raponi et al. [9].

Group-Testing Bayesian Optimization (GTBO) is a variable selection technique introduced in [6]. It assumes that the search space has active variable that affect the objective more than others. To identify

those variables it first goes through a group testing phase during which it selects groups of variables and tests their influence on the modeled function. Once it identifies the active variables it puts a prior on their length scales, attributing shorter length scales to the active variables, thus encouraging more sampling along these axes.

2.2 Implementation

The points for the design of experiments (DoE) used for DSP were drawn from a Sobol sequence. The GP described in 2.1 was implemented using BoTorch [2]. The optimization of the acquisition function was done using the BoTorch function that internally uses L-BFGS-B.

The DoE used in the original HeSBO code is latin hypercube sampling (LHS). The GP makes use of a Matérn kernel with $\nu = 5/2$ and ARD enabled. For the experiments, the lower dimensionality d was set to 25% of the full dimensionality of the problem, so $\lceil D/4 \rceil$. The acquisition function used was Expected Improvement. The acquisition function was optimized by drawing 2000 samples using LHS and picking the point with the highest EI. For DSP enhanced HeSBO (DSP-HeSBO) the only change that was made was the usage of the same GP as used by DSP, using the priors described in 1.

For PCA-BO the original implementation uses LHS for the DoE and uses a Matérn kernel with $\nu = 3/2$. α was set to 95% of variance explained for reducing the dimensionality. In contrast to the differential evolution suggested by the original paper, the acquisition function was optimized by drawing a random sample from a uniform random distribution and using BFGS to refine the solution and repeating this process for a set number of restarts. Due to limited documentation, the DSP enhanced version of PCA-BO had to implemented using slightly different methods. For the DSP enhanced version, SOBOL sampling was used for the DoE. The GP was again the same one used by the original DSP paper. The acquisition function was optimized by drawing 2000 points using LHS sampling, followed by using L-BFGS to refine the candidate solution. Exact implementations for DSP, HeSBO, PCABO and their DSP equivalents can be found in the following GitHub repository: https://github.com/Skippybal/DSPProject.

The DoE for GTBO is somewhat different as the D initial points are used to test the activeness of variables. The points sampled can be guided by a property of the benchmark. But by default GTBO mostly samples points close the center of the search space and varying a subset of variables at a time (for more details please refer to [6]). This implementation reused the original paper's code available at https://github.com/gtboauthors/gtbo. Almost all the changes made to the original code was setting variables that were originally contained in configuration files in the code directly. Thus, this implementation uses a Matérn kernel with a log normal prior on the lengthscales. By default these priors have scale 1. For active dimensions they have loc 1 and loc 3 for inactive variables. To combine this with the DSP the locs were increased by $\log(D)/2$. GTBO also uses a gamma output scale prior with a concentration of 2 and a rate of 0.15. The algorithm is allowed D initial points and set the maximum number of evaluations to $10 \times D$. The number of iterations after which to retrain the Gaussian process was set to $\lfloor D/2 \rfloor$ as this is a close match to what was in the default configuration available in the authors' repository. Because the implementation of GTBO has a minimum dimensionality of 6 it had to be excluded from the 2d benchmarks.

For REmBO, the code made available by the authors was in matlab, making it impractical for the benchmarks used here. Thus, a python implementation was made. This implementation used Botorch,

and the Adaptive Experimentation Platform (ax platform)[1] for the Bayesian Optimization part. On the other hand, the linear embedding was built using numpy. A SOBOL sequence was used for DoE as with the other methods. The surrogate used a Radial Basis Function kernel and a log expected improvement acquisition function. The code used here for REmBO and GTBO is available at https://github.com/Nael3004/BoProject. For the sake of consistency the same embedding dimension was used for REmBO as for HeSBO, $max(\lfloor D/4 \rfloor, 1)$. This allows it to still work with standard Bayesian optimization dimensionalities in the embedding space for all the experiments performed here. The vanilla version's log normal prior's loc on lengthscales was set to 1 and the scale used was $\sqrt{3}$. For the DSP version the loc was increased by $\log(D)/2$

2.3 Experimental setup

For the experimental setup, the BBOB [5] test suite was used. The function used for testing where: F1 (Sphere function), F8 (Rosenbrock Function), F12 (Ben Cigar function), F15 (Rastrigin function) and F21 (Gallagher's Gaussian 101-me peaks function). These functions where accessed through IoH [4]. The dimensionalities (D) that were tested were: 2, 10 and 40. For each experiment, the size of the DoE was set to be equal to the dimensionality of the function [10]. The minimum number of function evaluations for each experiment was set to $10 \cdot D$. Every experiment was repeated 5 times.

3 Results

When looking at the two-dimensional results 1, an interesting observation can be made. On F1 and F21 DSP outperforms the other algorithm by quite a large margin. On F12 DSP-PCABO and PCABO find better solutions than DSP, but converge slightly slower. DSP-PCABO also converges much slower than PCABO, but does find a better solution. On F8 PCABO and DSP give similar results, while HeSBO, DSP-HeSBO and DSP-PCABO perform slightly worse. On the other hand REmBO and DSP-REmBO perform quite well but have very similar results to each other.

When increasing the dimensionality to 10 dimensions, a very clear pattern emerges 2. On F1, F8, F15 and F21 the default DSP algorithm performs much better than all the algorithm that make use of linear embedding. This indicates that here the dimension reduction significantly hinders optimization. Another observation that can be made is that DSP-PCABO seems to perform equal or slightly worse than PCABO on these functions. On F1 GTBO and DSP-GTBO suddenly go through very steep improvements, this can be attributed to it being unable to properly fit a Gaussian Process using the active variables it identified. They then reset all lengthscales and perform Bayesian optimization, surpassing even DSP. It can also be noted that on F8, DSP-GTBO converges faster than vanilla GTBO and also finds a better solution, even achieving a slightly better performance than DSP. DSP-HeSBO tends to outperform HeSBO on most functions, even if only by a small margin. The most interesting function here is F12, as DSP no longer outperforms the other algorithms. Here both GTBO and DSP-GTBO achieve the best scores, while also converging the fastest. It can also be seen that PCA-BO performs better than both REmBO, HeSBO and DSP. DSP-PCABO also achieves a better score on average than PCA-BO, indicating that the DSP enhancement does improve performance in this scenario. On F15 while DSP still performs best, GTBO reaches closer performance than the other algorithms.

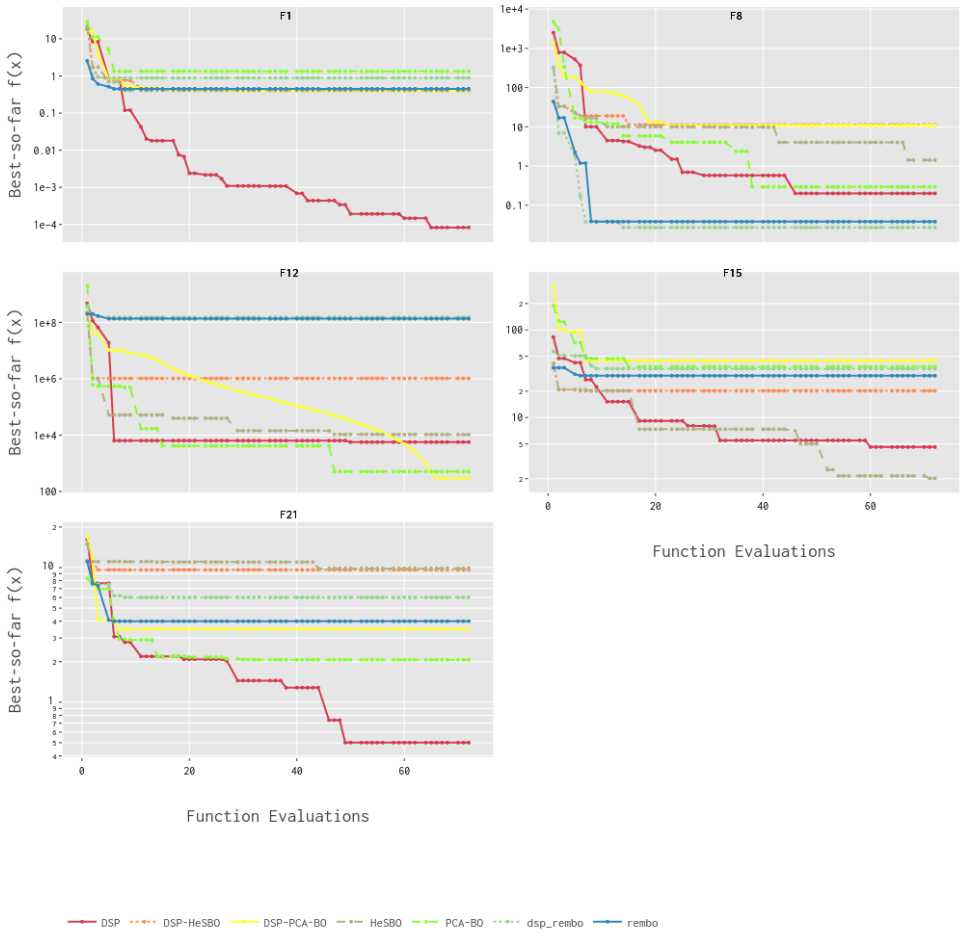

Figure 1: Results of algorithms on the 2-dimensional problems.

The results for the 40 dimensional problems 3 are similar to those of the 10 dimensional problems with DSP significantly outperforming the other methods on F1, F8, F15 and F21. Again PCA-BO achieves a better score on F12, with DSP-PCABO achieving similar performance. What is however different is that DSP-HeSBO consistently outperforms default HeSBO on every function. For PCABO, the DSP variant only finds worse results on F18, while on F15 and F21 it converges faster, but to a slightly worse performance on F21. These results are however overshadowed by the sheer magnitude of the difference between these algorithms and DSP on most functions. GTBO results are omitted as the full benchmarking could not be finished in time.

4 Discussion

This study was severely limited by the computational resources available. DSP is quite an expensive algorithm to run, as both fitting the GP with MAP estimation and optimizing the acquisition function are expensive operations that drastically increase in computational complexity as both the number of dimensions and the number of samples necessary increases. This prevented the testing of dimensions higher than 40, likely skewing results, as most methods proposed here are made specifically for high

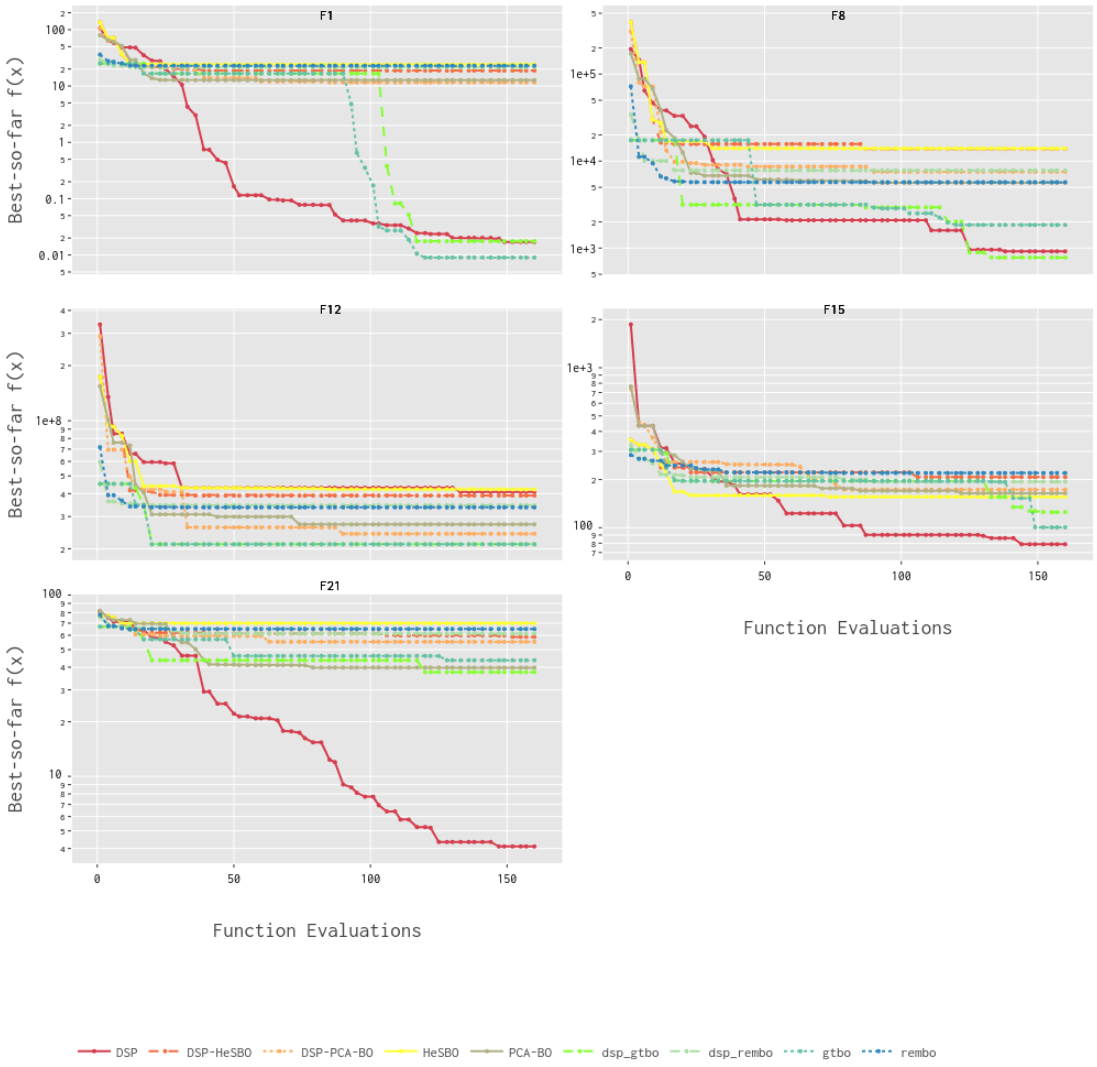

Figure 2: Results of algorithms on the 10 dimensional problems.

dimensional BO.

Other algorithms such as GTBO also showed to be hard to scale to higher dimensions in terms of computation time. 40 dimensional benchmarks of GTBO were attempted, but could not be finished in time. It is expected that DSP-GTBO would have the advantage on GTBO in 40 dimensions as it could cope with the increase in group sizes better. It is also expected that both algorithms would perform particularly well on F12 and possibly remain competitive on F8 and F15 (as they did in 10d).

Results would have likely shown a bigger increase in performance for both HeSBO and PCA-BO had the functions been tested on 100 dimensional problems. HeSBO already shows an increase, but this would have likely been increased in even higher dimensions. The major benefit of using DSP with HeSBO is the fact that the dimensionality of the active subspace can be kept relatively high, resulting in less information being lost by the embedding. PCA-BO would likely also show an increase as in 100 dimensions, it is more likely that the subspace found will include more than 20 principal components,

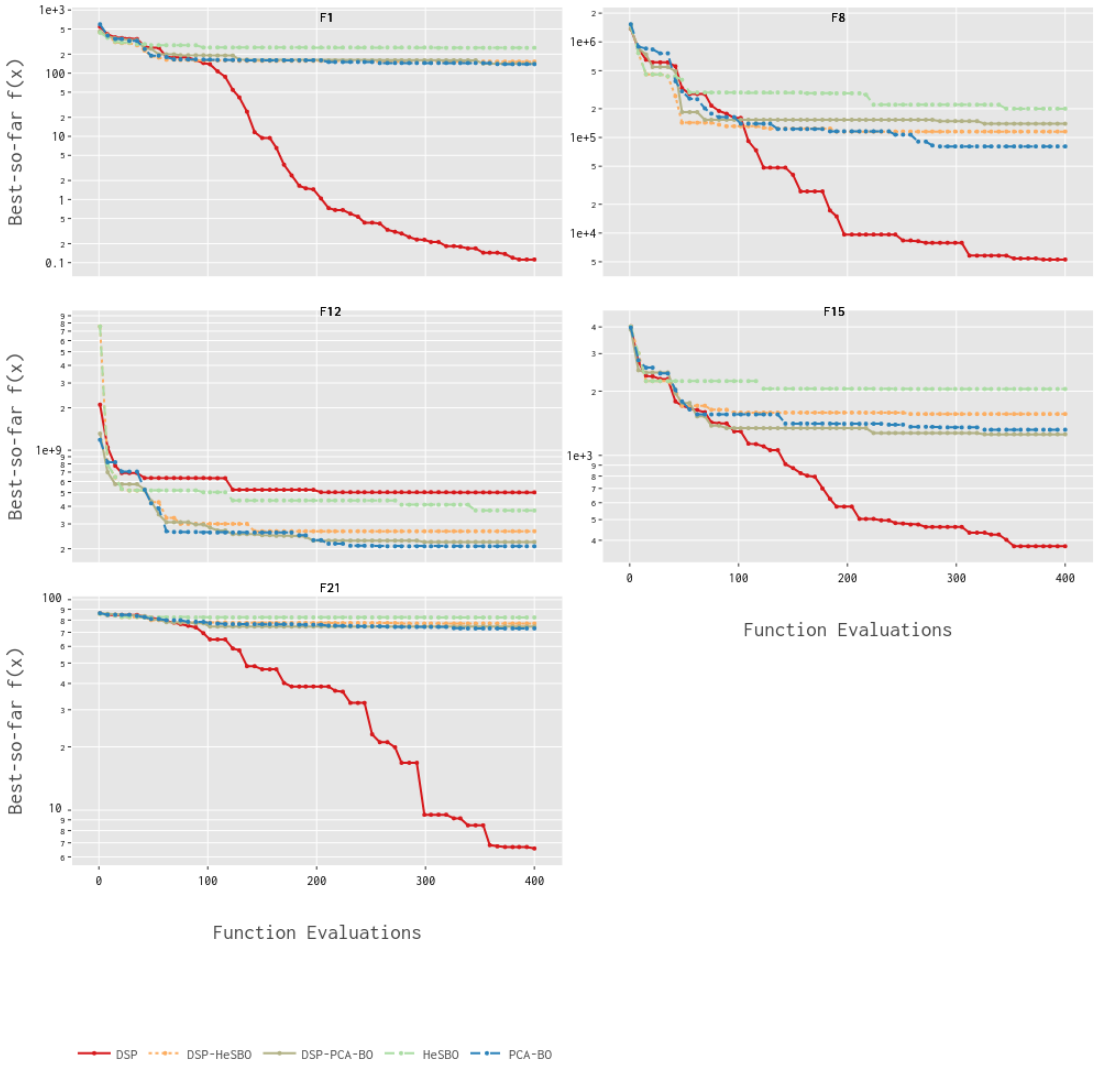

Figure 3: Results of algorithms on the 40 dimensional problems.

which in and of itself is high dimensions in the context of BO. In such a scenario using DSP would likely help increase performance. An interesting feature of DSP applied to these dimensionality reduction techniques is that it would allow for a more aggressive setting of the intrinsic dimensionality because of its good convergence speed even in higher dimensions. However, this couldn't be tested here because of the constraints previously mentioned.

This project also highlighted the importance of thorough code base documentation. Many of the changes implemented were relatively small but were made quite difficult as some algorithms tested have large and complex code with quite a scarce documentation. This can hinder improvements over previous algorithms.

Further benchmarks on functions with low intrinsic dimensionality may have interesting results as they fit the assumptions behind the dimensionality reduction algorithms better. Functions such as Levy or Hartmann would be good candidates to see how adding DSP to these dimensionality reduction

techniques changes their performances as the intrinsic dimensionality gets higher (¿20).

5 Conclusions

The idea of scaling priors to be proportional to the increase in the average distance increase caused by dimensionality is and effective approach for high dimensional Bayesian optimization. It consistently outperforms other methods reliant on linear embedding, except for on functions with high conditioning and are unimodal (F12). The addition of dimensional reduction techniques to DSP does not yield an increase in performance, and in most cases actually hinders performance drastically. The addition of these priors is also not something that is suitable for all algorithm. There are algorithms that will benefit, such as HeSBO, but other such as PCA-BO will see no consistent or significant increase in performance.

Similarly, for variable selection algorithms the addition of DSP does not seem to consistently increase performances. While the DSP-GTBO does outperform vanilla GTBO on some benchmarks, this improvement is neither large nor consistent enough to be attributed to the addition of DSP. DSP-GTBO does also outperform DSP on some benchmarks but when considering the performance of vanilla GTBO, it can be concluded that this is more due to GTBO's algorithm performing well than the combination improving over simple DSP.

References

[1] Eytan Bakshy, Lili Dworkin, Brian Karrer, Konstantin Kashin, Benjamin Letham, Ashwin Murthy, and Shaun Singh. Ae: A domain-agnostic platform for adaptive experimentation, 2018.

[2] Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization. In *Advances in Neural Information Processing Systems 33*, 2020.

[3] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. *Theoretical Computer Science*, 312(1):3–15, 2004.

[4] Carola Doerr, Hao Wang, Furong Ye, Sander van Rijn, and Thomas Bäck. IOHprofiler: A Benchmarking and Profiling Tool for Iterative Optimization Heuristics. *arXiv e-prints:1810.05281*, October 2018.

[5] Ouassim Elhara, Konstantinos Varelas, Duc Nguyen, Tea Tusar, Dimo Brockhoff, Nikolaus Hansen, and Anne Auger. Coco: the large scale black-box optimization benchmarking (bbob-largescale) test suite. *arXiv preprint arXiv:1903.06396*, 2019.

[6] Erik Orm Hellsten, Carl Hvarfner, Leonard Papenmeier, and Luigi Nardi. High-dimensional bayesian optimization with group testing, 2023.

[7] Carl Hvarfner, Erik Orm Hellsten, and Luigi Nardi. Vanilla bayesian optimization performs great in high dimension. *arXiv preprint arXiv:2402.02229*, 2024.

[8] Amin Nayebi, Alexander Munteanu, and Matthias Poloczek. A framework for bayesian optimization in embedded subspaces. In *International Conference on Machine Learning*, pages 4752–4761. PMLR, 2019.

[9] Elena Raponi, Hao Wang, Mariusz Bujny, Simonetta Boria, and Carola Doerr. High dimensional bayesian optimization assisted by principal component analysis. In *Parallel Problem Solving from Nature–PPSN XVI: 16th International Conference, PPSN 2020, Leiden, The Netherlands, September 5-9, 2020, Proceedings, Part I 16*, pages 169–183. Springer, 2020.

[10] Maria Laura Santoni, Elena Raponi, Renato De Leone, and Carola Doerr. Comparison of high-dimensional bayesian optimization algorithms on bbob. *ACM Transactions on Evolutionary Learning*, 4(3):1–33, 2024.

[11] Ziyu Wang, Frank Hutter, Masrour Zoghi, David Matheson, and Nando de Freitas. Bayesian optimization in a billion dimensions via random embeddings, 2016.